



INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DA PARAÍBA
CAMPUS CAMPINA GRANDE
COORDENAÇÃO DO CURSO SUPERIOR BACHARELADO EM ENGENHARIA DE
COMPUTAÇÃO

GABRIEL DOMINGOS DE MEDEIROS

**IMPLEMENTAÇÃO E VALIDAÇÃO DE UMA MÁQUINA VIRTUAL DE LÓGICA LADDER E
AGNÓSTICA DE HARDWARE BASEADA EM MODEL-BASED DESIGN**

GABRIEL DOMINGOS DE MEDEIROS

**IMPLEMENTAÇÃO E VALIDAÇÃO DE UMA MÁQUINA VIRTUAL DE LÓGICA LADDER E
AGNÓSTICA DE HARDWARE BASEADA EM MODEL-BASED DESIGN**

Monografia apresentada à Coordenação do
Curso de Engenharia de Computação do IFPB
- Campus Campina Grande, como requisito
parcial para conclusão do curso de Bacharelado em Engenharia de Computação.

Instituto Federal da Paraíba

Orientador: Fagner de Araújo Pereira

Campina Grande - PB
2026

GABRIEL DOMINGOS DE MEDEIROS

**IMPLEMENTAÇÃO E VALIDAÇÃO DE UMA MÁQUINA VIRTUAL DE LÓGICA LADDER E
AGNÓSTICA DE HARDWARE BASEADA EM MODEL-BASED DESIGN**

Monografia apresentada ao Curso de Bacharelado em Engenharia de Computação, do Instituto Federal de Educação, Ciência e Tecnologia da Paraíba - Campus Campina Grande, em cumprimento às exigências parciais para a obtenção do título de Bacharel em Engenharia da Computação.

Trabalho aprovado. Campina Grande - PB, _____ de _____
de 2026.

FAGNER DE ARAÚJO PEREIRA
Orientador

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
Membro da Banca

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
Membro da Banca

Campina Grande - PB
2026

Agradecimentos

XXXXXXXXXXXXXXXXXXXXXXX

Dedico este trabalho à minha família, pelo apoio incondicional, e a todos que, de alguma forma, contribuíram para que este momento fosse possível.

Alguma frase/pensamento (OPCIONAL)

Autor

Resumo

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX.

Palavras-chave: XXXXXXXXXXXXXXXXXXXX.

Abstract

XX.

Keywords: XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX.

Lista de Figuras

Lista de Tabelas

Lista de Abreviaturas

ARM	<i>Advanced RISC Machine</i>
CLP	Controlador Lógico Programável
FBD	<i>Function Block Diagram</i>
GPIO	<i>General Purpose Input/Output</i>
HAL	<i>Hardware Abstraction Layer</i>
IEC	<i>International Electrotechnical Commission</i>
IL	<i>Instruction List</i>
IoT	<i>Internet of Things</i>
ISA	<i>Instruction Set Architecture</i>
LD	<i>Ladder Diagram</i>
POSIX	<i>Portable Operating System Interface</i>
RAM	<i>Random Access Memory</i>
SFC	<i>Sequential Function Chart</i>
ST	<i>Structured Text</i>
VM	<i>Virtual Machine</i> (Máquina Virtual)
vPLC	<i>Virtual Programmable Logic Controller</i>

Sumário

1	Introdução	1
1.1	Contextualização e Descrição do Problema	1
1.2	Objetivos	2
1.2.1	Objetivo Geral	2
1.2.2	Objetivos Específicos	2
1.3	Justificativa	3
1.4	Estrutura do Documento	3
2	Fundamentação Teórica	5
2.1	O Tema: Automação Industrial, CLPs e Máquinas Virtuais	5
2.2	O Problema à Luz da Literatura	6
2.3	A Lacuna	7
2.4	Fundamentos da Solução	8
2.5	Síntese	9
3	Considerações Finais e Sugestões para Trabalhos Futuros	10
3.1	Sugestões para Trabalhos Futuros	10
A	Base de Dados	11
	Referências Bibliográficas	12

1 INTRODUÇÃO

1.1 CONTEXTUALIZAÇÃO E DESCRIÇÃO DO PROBLEMA

O mercado global de Controladores Lógicos Programáveis (CLPs) tem apresentado crescimento expressivo nos últimos anos, impulsionado pelos investimentos crescentes em automação industrial e pela modernização de sistemas de controle legados: estima-se que o setor movimenta atualmente cerca de US\$ 13,33 bilhões, com projeção de expansão contínua até 2031 [Mordor Intelligence 2026]. Esse crescimento reflete a consolidação da automação industrial como pilar central da Indústria 4.0, na qual controladores programáveis assumem papel crítico na interconexão de máquinas, sensores e processos produtivos. Nesse cenário, contudo, a expansão do setor convive com uma forte fragmentação tecnológica: a predominância de soluções proprietárias e de arquiteturas de *hardware* específicas por fabricante ainda limita a portabilidade de algoritmos de controle e favorece o aprisionamento tecnológico dos usuários finais. É nesse contexto que se insere a proposta deste trabalho: uma máquina virtual (VM) embarcada, capaz de interpretar instruções de lógica *Ladder* — baseada na norma IEC 61131-3 [International Electrotechnical Commission 2025] — de maneira independente da plataforma de *hardware* subjacente.

A lógica *Ladder* (*Ladder Diagram*) é uma das cinco linguagens de programação padronizadas pela IEC 61131-3 para controladores industriais, caracterizada pela representação gráfica de contatos e bobinas que remete aos antigos diagramas elétricos a relés, o que a tornou historicamente a linguagem mais difundida na automação industrial. Uma máquina virtual, por sua vez, corresponde a uma camada de *software* capaz de interpretar um conjunto de instruções (*bytecode*) de forma independente do *hardware* subjacente — de modo análogo ao papel desempenhado por máquinas virtuais de propósito geral em outros domínios da computação. Reunir esses dois conceitos exige a definição de uma Arquitetura de Conjunto de Instruções (*ISA — Instruction Set Architecture*) customizada e de tamanho fixo (32 *bits*), projetada para otimizar o processo de busca e decodificação (*fetch-decode*) e garantir o determinismo do tempo de varredura; a pesquisa foca especificamente na migração dessa arquitetura para microcontroladores da família *STM32* (núcleo *ARM Cortex-M7*), explorando técnicas de abstração de *hardware* (*HAL*) e metodologias de *Model-Based Design*. Propõe-se ainda a segregação estrutural entre o motor de execução (*firmware* estático) e a lógica de aplicação (*bytecode* tratado como dado), estabelecendo um ambiente de execução isolado (*sandbox*) que previne falhas críticas de processamento e viabiliza a gestão segura do ciclo de vida da aplicação industrial.

Apesar do crescimento do setor, a rigidez das arquiteturas convencionais de CLPs impõe barreiras significativas à inovação, dificultando a portabilidade de algoritmos de controle para plataformas de *hardware* mais modernas e acessíveis sem a necessidade de reengenharia completa do *firmware*. Além disso, a validação de lógicas de controle complexas diretamente

no *hardware* final apresenta riscos operacionais e custos elevados, carecendo de metodologias que integrem eficientemente a simulação computacional com a execução física.

No âmbito das soluções de código aberto atualmente disponíveis, como o ecossistema *OpenPLC* [Alves 2018], a abordagem predominante baseia-se na compilação de diagramas *Ladder* para código C nativo, seguida de compilação estática. Embora eficiente em termos de tempo de execução, essa metodologia funde a lógica de controle aos *drivers* de *hardware* em um binário monolítico, de modo que a adaptação para novas plataformas ou pinagens exige modificações diretas no código-fonte dos *drivers* em baixo nível, limitando drasticamente a portabilidade. Adicionalmente, a execução do código traduzido diretamente no processador elimina barreiras de proteção inerentes a sistemas operacionais: um erro na modelagem da lógica do usuário pode resultar em falhas críticas do sistema, comprometendo a estabilidade do equipamento industrial.

Por outro lado, abordagens alternativas baseadas em interpretadores de linguagens de *script* genéricas, como o *MicroPython* [MicroPython 2025], falham em atender aos requisitos de tempo real estrito da automação industrial, devido à gestão não determinística de memória (*Garbage Collection*), que introduz variações inaceitáveis no tempo de varredura. Diante do exposto, o problema técnico enfrentado por este trabalho consiste na ausência de uma camada de virtualização eficiente e aberta que permita a execução segura e determinística de lógicas de controle padronizadas em microcontroladores de propósito geral, garantindo a fidelidade comportamental entre o modelo projetado em ambiente de simulação e o sistema embarcado final.

1.2 OBJETIVOS

1.2.1 Objetivo Geral

Diante do problema apresentado, este trabalho tem como objetivo geral desenvolver e validar uma máquina virtual embarcada capaz de interpretar *bytecode* de lógica *Ladder* com fidelidade semântica à norma IEC 61131-3, executando de forma determinística e agnóstica de *hardware* em um microcontrolador da família *STM32*.

1.2.2 Objetivos Específicos

- Especificar e documentar formalmente uma Arquitetura de Conjunto de Instruções (*ISA*) de 32 *bits*, compatível com as primitivas operacionais do subconjunto *Ladder Diagram* da IEC 61131-3;
- Modelar lógicas de controle de referência em ambiente *Simulink/Stateflow*, segundo a metodologia de *Model-Based Design*;
- Implementar, em linguagem C, o motor de execução (*VM*) responsável pelo ciclo de

busca, decodificação e execução do *bytecode*;

- Desenvolver uma Camada de Abstração de *Hardware* (*HAL*) para o microcontrolador *STM32F7* (núcleo *ARM Cortex-M7*);
- Validar cruzadamente o comportamento da *VM* embarcada frente ao modelo de referência simulado, comparando as tabelas de imagem de saída para um mesmo conjunto de estímulos de entrada;
- Analisar quantitativamente o desempenho da solução, mensurando o tempo de varredura (*scan cycle*) e o consumo de memória *RAM* e *Flash*.

1.3 JUSTIFICATIVA

A relevância deste trabalho decorre da lacuna identificada entre as soluções de código aberto disponíveis e os requisitos de portabilidade e determinismo da automação industrial. Ao segregar o motor de execução da lógica de aplicação, a arquitetura proposta elimina a necessidade de recompilação do *firmware* a cada alteração da lógica *Ladder*, reduzindo o tempo de comissionamento e o risco de falhas decorrentes de atualizações completas de memória. O caráter de ambiente isolado (*sandbox*) confere ainda uma camada de segurança inexistente nas soluções que traduzem a lógica do usuário diretamente para código nativo, contendo eventuais erros estruturais antes que comprometam a estabilidade do sistema embarcado.

Do ponto de vista social e econômico, a proposta de uma solução aberta e portátil para múltiplas plataformas de *hardware* contribui para reduzir o aprisionamento tecnológico a fabricantes específicos, democratizando o acesso a tecnologias de automação hoje concentradas em soluções proprietárias de alto custo.

1.4 ESTRUTURA DO DOCUMENTO

A metodologia adotada caracteriza-se como pesquisa aplicada de natureza experimental e abordagem predominantemente quantitativa, estruturada em etapas sequenciais com entregas incrementais que partem da especificação formal da arquitetura até a validação funcional integrada ao *hardware*; o detalhamento de cada etapa — especificação da *ISA*, modelagem de referência, implementação do motor de execução, desenvolvimento da *HAL*, integração com *hardware*, validação cruzada e análise de desempenho — será apresentado nos capítulos de desenvolvimento deste trabalho.

O restante deste trabalho está organizado da seguinte forma: o Capítulo 2 apresenta a fundamentação teórica, contemplando a norma IEC 61131-3, os trabalhos relacionados e o estado da prática em soluções para execução de lógicas de controle industrial, além dos conceitos de arquitetura que sustentam a solução proposta. Os capítulos subsequentes

apresentarão a concepção e a modelagem detalhada da solução, os resultados obtidos na validação cruzada e na análise de desempenho, seguidos das considerações finais e das sugestões para trabalhos futuros, no Capítulo 3.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 O TEMA: AUTOMAÇÃO INDUSTRIAL, CLPS E MÁQUINAS VIRTUAIS

Para compreender o problema investigado, é necessário situar dois conceitos centrais: o Controlador Lógico Programável, unidade fundamental da automação industrial, e a máquina virtual, abstração computacional que sustenta a proposta deste trabalho.

Um Controlador Lógico Programável (CLP) é um dispositivo digital programável projetado para controlar processos industriais por meio da execução cíclica de um programa de controle, em um processo denominado ciclo de varredura (*scan cycle*): a cada ciclo, o CLP lê o estado das entradas físicas, executa a lógica de controle armazenada em memória e atualiza o estado das saídas [Bolton 2015]. A previsibilidade da duração desse ciclo é um requisito central da automação industrial, uma vez que atrasos ou variações no tempo de varredura podem comprometer a segurança e a correção do processo controlado.

A norma IEC 61131-3, cuja quarta edição foi publicada em maio de 2025 pela Comissão Eletrotécnica Internacional (IEC), constitui a norma internacional que define a arquitetura de *software* e as linguagens de programação para controladores programáveis [International Electrotechnical Commission 2025]. A norma especifica a sintaxe e a semântica de um conjunto unificado de linguagens, incluindo *Structured Text (ST)*, *Ladder Diagram (LD)*, *Function Block Diagram (FBD)* e *Sequential Function Chart (SFC)*; a edição mais recente removeu a linguagem *Instruction List (IL)*, anteriormente marcada como obsoleta. A importância da norma para este trabalho é fundamental, pois a *ISA* customizada da máquina virtual proposta é projetada para representar fielmente as primitivas operacionais definidas no subconjunto *LD* da IEC 61131-3, garantindo compatibilidade semântica com o padrão industrial.

Uma máquina virtual (*VM*), por sua vez, corresponde a uma camada de *software* que reproduz, total ou parcialmente, a interface de uma máquina física ou abstrata, permitindo a execução de um programa — tipicamente representado como *bytecode* — de forma independente da plataforma de *hardware* subjacente [Smith e Nair 2005]. Essa independência é obtida por meio de um interpretador que traduz as instruções do *bytecode* em operações concretas sobre o *hardware* hospedeiro, segregando a lógica da aplicação da implementação do motor de execução.

A convergência desses dois conceitos — o CLP como unidade de controle cíclico e determinístico, e a *VM* como camada de abstração portátil — fundamenta a proposta central deste trabalho: uma máquina virtual embarcada capaz de interpretar *bytecode* de lógica *Ladder*, compatível com a IEC 61131-3, preservando tanto o determinismo exigido pela automação industrial quanto a portabilidade típica das arquiteturas virtualizadas.

2.2 O PROBLEMA À LUZ DA LITERATURA

A Introdução deste trabalho identificou a fragmentação tecnológica e o aprisionamento a soluções proprietárias como obstáculos centrais à portabilidade de lógicas de controle industrial. A literatura científica corrobora e aprofunda essa constatação, tanto do ponto de vista teórico do determinismo temporal quanto das tentativas já realizadas de virtualizar o modelo IEC 61131-3.

Do ponto de vista teórico, um sistema de tempo real é definido não apenas pela correção lógica de seus resultados, mas também pela correção temporal: uma resposta correta produzida fora do prazo estabelecido é considerada uma falha do sistema [Liu 2000]. Essa exigência de determinismo temporal impõe restrições severas sobre qualquer camada de *software* interposta entre a lógica de controle e o *hardware* — entre elas, uma máquina virtual —, pois cada etapa adicional de interpretação introduz uma fonte potencial de variabilidade (*jitter*) no tempo de execução. É precisamente essa tensão entre portabilidade e determinismo que a literatura sobre virtualização de CLPs busca resolver.

As primeiras propostas de virtualização do modelo IEC 61131-3 datam de mais de uma década. Zhou e Chen [Zhou e Chen 2009] propuseram uma máquina virtual para CLPs utilizando a linguagem *Instruction List (IL)* — posteriormente removida da norma — como código intermediário, demonstrando a viabilidade de um motor de interpretação dedicado, ainda sem validação em *hardware* embarcado de recursos limitados. Na mesma direção, Plaza e Medrano [Plaza e Medrano 2006] demonstraram que o modelo IEC 61131-3 pode ser mapeado para primitivas *POSIX* (*Pthreads* e temporizadores de alta resolução) sob o *RTLinux*, obtendo desempenho comparável a produtos comerciais; a abordagem, no entanto, permanece vinculada a plataformas com sistema operacional de tempo real, não contemplando a execução *bare-metal* em microcontroladores.

Mais recentemente, Sadolewski e Trybus desenvolveram um ambiente de controle multi-plataforma cujo compilador e máquina virtual foram efetivamente testados em um microcontrolador *STM32* de núcleo *ARM* [Sadolewski e Trybus 2022] e, em trabalho subsequente, propuseram um modelo semântico executável para verificar formalmente a corretude do *runtime* da *VM* frente à especificação da linguagem [Sadolewski e Trybus 2024]. Esses dois trabalhos são os mais próximos, na literatura, da proposta aqui apresentada: ambos validam a viabilidade de uma *VM* para IEC 61131-3 em *hardware* embarcado real. Entretanto, o foco dos autores recai sobre a linguagem *Structured Text* e sobre a verificação formal da semântica de execução, não sobre o subconjunto *Ladder Diagram* nem sobre a validação cruzada contra um modelo comportamental de referência simulado — lacuna que o presente trabalho busca preencher.

No âmbito das soluções de código aberto amplamente adotadas, Alves [Alves 2018] apresentou o *OpenPLC*, evidenciando que sua compilação estática via *MatIEC* funde a lógica do usuário ao binário do *firmware*, eliminando a possibilidade de atualização dinâmica da lógica de controle. Essa limitação é estrutural, não incidental: qualquer abordagem baseada em

compilação — em vez de interpretação — da lógica de controle tende a acoplar aplicação e motor de execução, comprometendo a portabilidade e a segurança do sistema diante de erros de modelagem.

Por fim, a metodologia de *Model-Based Design* tem sido empregada como estratégia de validação de lógicas de controle: Duggan et al. [Duggan *et al.* 2022] demonstram que a documentação de validação e o código *PLC* podem ser gerados automaticamente a partir de um modelo simulado, enquanto a plataforma *Juliet & Romeo*, da *Cognibotics* [Cognibotics 2025], evidencia, em escala industrial de maior porte, que a separação entre uma linguagem de alto nível e uma máquina virtual de execução determinística é uma estratégia arquitetural viável. Considerados em conjunto, esses trabalhos confirmam tanto a viabilidade técnica da virtualização de lógicas de controle quanto a ausência recorrente de soluções voltadas especificamente a microcontroladores de recursos limitados com validação cruzada explícita contra um modelo de referência simulado.

2.3 A LACUNA

Ao lado dos trabalhos acadêmicos, o estado da prática em soluções comerciais e de código aberto reforça a mesma lacuna sob uma perspectiva de mercado. O *OpenPLC* [OpenPLC Project 2025], solução de código aberto mais madura para CLPs em *hardware* de propósito geral, mantém a limitação estrutural já discutida: a lógica do usuário funde-se ao *firmware* compilado, exigindo regravação completa a cada alteração. O *CODESYS Runtime* [CODESYS Group 2025], por sua vez, resolve o problema da portabilidade ao gerar código nativo para uma ampla gama de processadores, mas o faz como produto comercial de código fechado, cujo custo de licenciamento e cuja dependência de um *Runtime Toolkit* proprietário inviabilizam a adaptação livre para novo *hardware*. O *Simulink PLC Coder* [MathWorks 2025] demonstra, na prática, a viabilidade de gerar código IEC 61131-3 automaticamente a partir de um modelo — corroborando a metodologia de *Model-Based Design* discutida anteriormente —, mas pressupõe a existência de um CLP alvo convencional, não oferecendo camada de virtualização própria para microcontroladores. Por fim, abordagens genéricas como o *MicroPython* [MicroPython 2025] mostram que interpretadores de *bytecode* são viáveis em microcontroladores de baixo custo, mas sua gestão de memória por *Garbage Collection* introduz pausas não determinísticas incompatíveis com o ciclo de varredura industrial.

Em conjunto, a literatura acadêmica e o estado da prática convergem para a mesma constatação: nenhuma das soluções examinadas concilia, simultaneamente, (i) execução em microcontroladores de recursos limitados, (ii) determinismo temporal compatível com a automação industrial, (iii) segregação entre motor de execução e lógica de aplicação que permita atualização dinâmica segura, e (iv) validação cruzada explícita contra um modelo de referência simulado. É essa lacuna — confirmada tanto pelos trabalhos de Sadolewski e Trybus [Sadolewski e Trybus 2022; Sadolewski e Trybus 2024] quanto pelas limitações do *OpenPLC* [OpenPLC Project 2025], do *CODESYS* [CODESYS Group 2025] e do *MicroPython* [MicroPython

2025] — que justifica e delimita a proposta deste trabalho.

2.4 FUNDAMENTOS DA SOLUÇÃO

Diante da lacuna identificada, esta seção apresenta os fundamentos teóricos que sustentam a solução proposta: uma máquina virtual embarcada baseada em Arquitetura de Conjunto de Instruções (*ISA*) customizada, Camada de Abstração de *Hardware* (*HAL*) e ambiente de execução isolado (*sandbox*), validada por meio de *Model-Based Design*.

Uma *ISA* define a interface entre o *software* e o *hardware* de um processador ou máquina virtual, especificando o formato, a semântica e o repertório das instruções executáveis [Patterson e Hennessy 2020]. *ISAs* de largura fixa — nas quais toda instrução ocupa o mesmo número de *bits* — simplificam o estágio de busca e decodificação (*fetch-decode*), pois o avanço do ponteiro de instrução é constante e o tempo de decodificação torna-se previsível, característica desejável para sistemas com requisitos de determinismo temporal, como o ciclo de varredura de um CLP. O ciclo *fetch-decode-execute* constitui o modelo clássico de operação de um processador ou máquina virtual: a instrução é buscada na memória de programa, decodificada para identificar a operação e os operandos, e então executada, repetindo-se o processo para a instrução seguinte.

A *HAL* é um padrão arquitetural que encapsula o acesso aos periféricos físicos de um dispositivo — como *GPIOs*, temporizadores e conversores analógico-digitais — por trás de uma interface estável e independente do *hardware* subjacente. A adoção de uma *HAL* permite que a lógica de aplicação e o motor de execução permaneçam inalterados quando o projeto é portado para uma nova plataforma, restringindo o esforço de adaptação à reimplementação da própria camada.

Um ambiente de execução isolado (*sandbox*) é um mecanismo que restringe o escopo de atuação de um programa em execução, validando em tempo real os limites de seus acessos — como endereços de memória e índices de entrada e saída — e rejeitando operações inválidas. Em sistemas embarcados críticos, esse mecanismo contém erros estruturais na lógica do usuário antes que eles comprometam a estabilidade do restante do sistema, respondendo diretamente à limitação do *OpenPLC* discutida na seção anterior.

Por fim, o *Model-Based Design* é uma metodologia de desenvolvimento na qual um modelo executável do sistema — e, quando aplicável, da planta controlada — é utilizado como artefato central ao longo de todo o ciclo de vida do projeto, da especificação à verificação [Nicolescu e Mosterman 2009]. Nesse paradigma, a simulação do modelo serve como referência comportamental para validar a implementação final: estratégia empregada por Duggan et al. [Duggan et al. 2022] na geração e validação automática de código *PLC*, e adotada neste trabalho para a validação cruzada entre a lógica de controle modelada em *Simulink/Stateflow* e sua execução na máquina virtual embarcada — respondendo, assim, à ausência de validação cruzada sistemática identificada nos trabalhos de Sadolewski e Trybus.

2.5 SÍNTESE

Os fundamentos apresentados neste capítulo — da norma IEC 61131-3 e do conceito de máquina virtual, passando pela literatura sobre determinismo temporal e pelas tentativas anteriores de virtualização do modelo IEC 61131-3, até os princípios arquiteturais de *ISA*, *HAL*, *sandbox* e *Model-Based Design* — sustentam de forma articulada a solução proposta. Onde a literatura acadêmica demonstrou viabilidade técnica sem validação cruzada sistemática contra um modelo de referência, e onde o estado da prática ofereceu portabilidade ou determinismo, mas raramente ambos de forma simultânea em *hardware* de recursos limitados, este trabalho posiciona-se para preencher essa lacuna: uma máquina virtual embarcada, agnóstica de *hardware*, para lógica *Ladder* compatível com a IEC 61131-3.

3 CONSIDERAÇÕES FINAIS E SUGESTÕES PARA TRABALHOS FUTUROS

3.1 SUGESTÕES PARA TRABALHOS FUTUROS

A BASE DE DADOS

Capítulo opcional usado para que o autor coloque um texto, documento ou informação referente ao assunto do trabalho, mas que não deve interromper a leitura.

Referências Bibliográficas

- [Alves 2018] ALVES, T. OpenPLC: An IEC 61131-3 compliant open source industrial controller for cyber security research. *Computers & Security*, v. 78, p. 364–379, 2018. 2, 6
- [Bolton 2015] BOLTON, W. *Programmable Logic Controllers*. 6. ed. [S.l.]: Newnes, 2015. 5
- [CODESYS Group 2025] CODESYS Group. *CODESYS Runtime: the adaptable runtime environment*. 2025. Disponível em: <<https://www.codesys.com/products/runtime/>>. Acesso em: 2025. 7
- [Cognibotics 2025] Cognibotics. *Juliet & Romeo® and IEC 61131: How They Work Together*. 2025. Disponível em: <<https://www.cognibotics.com/en/articles/juliet-romeo-and-iec-61131>>. Acesso em: 2025. 7
- [Duggan et al. 2022] DUGGAN, J. et al. A model-based approach to automated validation and generation of PLC code for manufacturing equipment in regulated environments. *Applied Sciences*, v. 12, n. 15, 2022. Art. 7506. 7, 8
- [International Electrotechnical Commission 2025] International Electrotechnical Commission. *IEC 61131-3:2025 — Programmable controllers — Part 3: Programming languages*. 4. ed. Genebra, 2025. 1, 5
- [Liu 2000] LIU, J. W. S. *Real-Time Systems*. [S.l.]: Prentice Hall, 2000. 6
- [MathWorks 2025] MathWorks. *Simulink PLC Coder: Generate IEC 61131-3 Structured Text and Ladder Diagrams*. 2025. Disponível em: <<https://www.mathworks.com/products/simulink-plc-coder.html>>. Acesso em: 2025. 7
- [MicroPython 2025] MicroPython. *MicroPython — Python for microcontrollers*. 2025. Disponível em: <<https://micropython.org/>>. Acesso em: 2025. 2, 7
- [Mordor Intelligence 2026] Mordor Intelligence. *Programmable Logic Controller (PLC) Market Size & Share Analysis — Growth Trends and Forecast (2026 - 2031)*. 2026. Disponível em: <<https://www.mordorintelligence.com/industry-reports/programmable-logic-controller-plc-market>>. Acesso em: 2026. 1
- [Nicolescu e Mosterman 2009] NICOLESCU, G.; MOSTERMAN, P. J. *Model-Based Design for Embedded Systems*. [S.l.]: CRC Press, 2009. 8
- [OpenPLC Project 2025] OpenPLC Project. *OpenPLC Runtime v4*. 2025. Disponível em: <<https://github.com/Autonomy-Logic/openplc-runtime>>. Acesso em: 2025. 7
- [Patterson e Hennessy 2020] PATTERSON, D. A.; HENNESSY, J. L. *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*. 2. ed. [S.l.]: Morgan Kaufmann, 2020. 8
- [Plaza e Medrano 2006] PLAZA, I.; MEDRANO, C. Analysis and implementation of the IEC 61131-3 software model under POSIX real-time operating systems. *Microprocessors and Microsystems*, v. 30, n. 8, p. 497–508, 2006. 6

[Sadolewski e Trybus 2022] SADOLEWSKI, J.; TRYBUS, B. Compiler and virtual machine of a multiplatform control environment. *Bulletin of the Polish Academy of Sciences: Technical Sciences*, v. 70, n. 2, 2022. Art. e140554. DOI: 10.24425/bpasts.2022.140554. 6, 7

[Sadolewski e Trybus 2024] SADOLEWSKI, J.; TRYBUS, B. Verification of control system runtime using an executable semantic model. *Algorithms*, v. 17, n. 7, 2024. Art. 273. DOI: 10.3390/a17070273. 6, 7

[Smith e Nair 2005] SMITH, J. E.; NAIR, R. *Virtual Machines: Versatile Platforms for Systems and Processes*. [S.l.]: Morgan Kaufmann, 2005. 5

[Zhou e Chen 2009] ZHOU, C.; CHEN, H. Development of a PLC virtual machine orienting IEC 61131-3 standard. In: *International Conference on Measuring Technology and Mechatronics Automation (ICMTMA)*. [S.l.: s.n.], 2009. 6